



Machine Learning

MCA

(Practical User Manual)

Input Part

Step-1: The input data for the experiment are collected from online machine repository.

Step-2: In this research two datasets (iris dataset and Pima Indian diabetes dataset) are used for the experiment.

Step-3: The first dataset is small Iris dataset which contains five attributes (4 numeric and 1 predictive class attributes).

- i. The Iris dataset include class attribute and 150 numbers of instances.
- ii. The attributes are sepal length, petal length, sepal width, petal width.
- iii. The class attributes may be iris setosa, iris versicolor and iris virginica.
- iv. In this research only instances have been taken and two class attributes “iris setosa representing 1 and iris versicolor representing 0”.
- v. Iris dataset taken link: [Python - Basics of Pandas using Iris Dataset - GeeksforGeeks](#)

Step-4: The second dataset is large Pima Indian Diabetes dataset which contains 9 attributes (8 numeric and 1 predictive class attribute).

- i. The diabetes dataset include class attribute and 268 numbers of instances.
- ii. The attributes are Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Diabetes Pedigree Function, Age.
- iii. Pima Indian Diabetes dataset link-1: [ML | Data Preprocessing in Python - GeeksforGeeks](#)
- iv. Pima Indian Diabetes dataset link-2: [Pima Indians Diabetes Database \(kaggle.com\)](#)

- v. Pima Indian Diabetes dataset link-3: [Pima Indian Diabetes Data Analysis in Python | Canopus Business Management Group \(collaborat.com\)](https://canopusbusinessmanagementgroup.com/collaborat.com)

Output Part

Step-1: The output data for the experiment are produce by using Python IDLE and WEKA Tool.

Step-2: Installation

- i. To install Python and WAKA tool Latest Version.
- ii. Go to Command Prompt and run it as administrator.
- iii. By using command Prompt check python version as >>python -version, pip -version, pip list and so on.

Step-4: Install python module as per the need.(eg; pip install pandas)

Step-5: Import the Python library NumPy(Like >> import numpy).

Iris Datasets

Step-6: Reading the dataset “Iris.csv”.

>> data = pd.read_csv("give a downloaded dataset location ") → Using python

Eg; >> data=pd.read_csv("C:\\Users\\user\\Desktop\\Weka\\iris_csv.csv")

Eg; >> data=pd.read_csv("C:/Users/user/Desktop/Weka/iris_csv.csv") →Using Anaconda Navigator

i. Displaying up the top rows of the dataset with their columns.

```
>> data.head()
```

```
import pandas as pd
data=pd.read_csv("C:\\Users\\user\\Desktop\\Weka\\iris_csv.csv")
data.head()
   sepallength  sepalwidth  petallength  petalwidth    class
0           5.1         3.5         1.4         0.2  Iris-setosa
1           4.9         3.0         1.4         0.2  Iris-setosa
2           4.7         3.2         1.3         0.2  Iris-setosa
3           4.6         3.1         1.5         0.2  Iris-setosa
4           5.0         3.6         1.4         0.2  Iris-setosa
```

ii. Displaying the number of rows randomly.

```
>> data.sample()
```

```
data.sample()
   sepallength  sepalwidth  petallength  petalwidth    class
11           4.8         3.4         1.6         0.2  Iris-setosa
data.sample(10)
   sepallength  sepalwidth  petallength  petalwidth    class
147           6.5         3.0         5.2         2.0  Iris-virginica
125           7.2         3.2         6.0         1.8  Iris-virginica
137           6.4         3.1         5.5         1.8  Iris-virginica
7           5.0         3.4         1.5         0.2    Iris-setosa
144           6.7         3.3         5.7         2.5  Iris-virginica
85           6.0         3.4         4.5         1.6  Iris-versicolor
75           6.6         3.0         4.4         1.4  Iris-versicolor
5           5.4         3.9         1.7         0.4    Iris-setosa
124           6.7         3.3         5.7         2.1  Iris-virginica
111           6.4         2.7         5.3         1.9  Iris-virginica
```

iii. Displaying the number of columns, shape and names of the columns.

```
>> data.columns
```

```
>> data.shape
```

```
>> print(data)
```

```
data.columns
Index(['sepalength', 'sepalwidth', 'petallength', 'petalwidth', 'class'], dtype='object')
data.shape
(150, 5)
print(data)
   sepalength  sepalwidth  petallength  petalwidth      class
0          5.1         3.5         1.4         0.2  Iris-setosa
1          4.9         3.0         1.4         0.2  Iris-setosa
2          4.7         3.2         1.3         0.2  Iris-setosa
3          4.6         3.1         1.5         0.2  Iris-setosa
4          5.0         3.6         1.4         0.2  Iris-setosa
..         ...         ...         ...         ...         ...
145         6.7         3.0         5.2         2.3  Iris-virginica
146         6.3         2.5         5.0         1.9  Iris-virginica
147         6.5         3.0         5.2         2.0  Iris-virginica
148         6.2         3.4         5.4         2.3  Iris-virginica
149         5.9         3.0         5.1         1.8  Iris-virginica

[150 rows x 5 columns]
```

iv. Slicing the rows.

```
>> print(data[10:21])
```

```
>> sliced_data=data[10:21]
```

```
>> print(sliced_data)
```

```
sliced_data=data[10:21]
print(sliced_data)
```

	sepalength	sepalwidth	petallength	petalwidth	class
10	5.4	3.7	1.5	0.2	Iris-setosa
11	4.8	3.4	1.6	0.2	Iris-setosa
12	4.8	3.0	1.4	0.1	Iris-setosa
13	4.3	3.0	1.1	0.1	Iris-setosa
14	5.8	4.0	1.2	0.2	Iris-setosa
15	5.7	4.4	1.5	0.4	Iris-setosa
16	5.4	3.9	1.3	0.4	Iris-setosa
17	5.1	3.5	1.4	0.3	Iris-setosa
18	5.7	3.8	1.7	0.3	Iris-setosa
19	5.1	3.8	1.5	0.3	Iris-setosa
20	5.4	3.4	1.7	0.2	Iris-setosa

v. Displaying the specific rows using “iloc” and “loc” functions.

```
data.iloc[5]
sepalength      5.4
sepalwidth      3.9
petallength     1.7
petalwidth      0.4
class           Iris-setosa
Name: 5, dtype: object
data.loc[data["class"] = "iris-setosa"]
SyntaxError: cannot assign to subscript here. Maybe you meant '==' instead of '='?
data.loc[data["class"] == "iris-setosa"]
Empty DataFrame
Columns: [sepalength, sepalwidth, petallength, petalwidth, class]
Index: []
```

vi. Selecting rows using pandas .iloc and loc

```
>> row1 = data.loc[3]
```

```
row1 = data.loc[3]
row2 = data.iloc[3]
row1 == row2
sepallength    True
sepalwidth     True
petallength    True
petalwidth     True
class          True
Name: 3, dtype: bool
row1 = data.iloc[[4, 5, 6, 7]]
row2 = data.iloc[4:8]
row1 == row2
```

	sepallength	sepalwidth	petallength	petalwidth	class
4	True	True	True	True	True
5	True	True	True	True	True
6	True	True	True	True	True
7	True	True	True	True	True

vii. Counting the number of counts of unique values using “value_counts()”.

```
>> data[“class”].value_counts()
```

```
data[“class”].value_counts()
class
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

viii. Extracting minimum and maximum from a column.

```
>> min_data=data[“sepallength”].min()
```

```
>> max_data=data["sepalength"].max()
```

```
>> print("Minimum:",min_data, "\nMaximum:", max_data)
```

```
min_data=data["sepalength"].min()
max_data=data["sepalength"].max()
print("Minimum:",min_data, "\nMaximum:", max_data)
Minimum: 4.3
Maximum: 7.9
data.style
<pandas.io.formats.style.Styler object at 0x000002350D46FD10>
```

ix. Cleaning and detecting missing values

```
>> data.isnull()
```

```
data.isnull()
   sepalength  sepalwidth  petallength  petalwidth  class
0         False         False         False         False  False
1         False         False         False         False  False
2         False         False         False         False  False
3         False         False         False         False  False
4         False         False         False         False  False
..         ...         ...         ...         ...         ...
145        False         False         False         False  False
146        False         False         False         False  False
147        False         False         False         False  False
148        False         False         False         False  False
149        False         False         False         False  False

[150 rows x 5 columns]
```

```
IDLE Shell 3.11.3
File Edit Shell Debug Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>> import pandas as pd
>>> data=pd.read_csv("C:\\Users\\user\\Desktop\\Weka\\iris_csv.csv")
>>> data.head()
   sepalength  sepalwidth  petallength  petalwidth  class
0          5.1         3.5         1.4         0.2  Iris-setosa
1          4.9         3.0         1.4         0.2  Iris-setosa
2          4.7         3.2         1.3         0.2  Iris-setosa
3          4.6         3.1         1.5         0.2  Iris-setosa
4          5.0         3.6         1.4         0.2  Iris-setosa
>>> print data()
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
>>> print(data)
   sepalength  sepalwidth  petallength  petalwidth  class
0          5.1         3.5         1.4         0.2  Iris-setosa
1          4.9         3.0         1.4         0.2  Iris-setosa
2          4.7         3.2         1.3         0.2  Iris-setosa
3          4.6         3.1         1.5         0.2  Iris-setosa
4          5.0         3.6         1.4         0.2  Iris-setosa
..          ...         ...         ...         ...         ...
145         6.7         3.0         5.2         2.3  Iris-virginica
146         6.3         2.5         5.0         1.9  Iris-virginica
147         6.5         3.0         5.2         2.0  Iris-virginica
148         6.2         3.4         5.4         2.3  Iris-virginica
149         5.9         3.0         5.1         1.8  Iris-virginica

[150 rows x 5 columns]
>>> print(data[10:21])
   sepalength  sepalwidth  petallength  petalwidth  class
10          5.4         3.7         1.5         0.2  Iris-setosa
11          4.8         3.4         1.6         0.2  Iris-setosa
12          4.8         3.0         1.4         0.1  Iris-setosa
13          4.3         3.0         1.1         0.1  Iris-setosa
14          5.8         4.0         1.2         0.2  Iris-setosa
15          5.7         4.4         1.5         0.4  Iris-setosa
16          5.4         3.9         1.3         0.4  Iris-setosa
17          5.1         3.5         1.4         0.3  Iris-setosa
18          5.7         3.8         1.7         0.3  Iris-setosa
19          5.1         3.8         1.5         0.3  Iris-setosa
20          5.4         3.4         1.7         0.2  Iris-setosa
>>>
```

Similarly Pima Indian Diabetes Datasets

Step-7: Reading the dataset “diabetes.csv”.

```
>> data = pd.read_csv("give a downloaded dataset location ")
```

Eg; >> data=pd.read_csv("C:\\Users\\user\\Desktop\\Weka\\diabetes_csv.csv")

i. Displaying up the top rows of the dataset with their columns.

```
>> data.head()
```

```
import pandas as pd
data=pd.read_csv('C:\\Users\\user\\Desktop\\Weka\\diabetes_csv.csv')
data.head()
   Pregnancies  Glucose  BloodPressure  ...  DiabetesPedigreeFunction  Age  Outcome
0            4      183             0  ...                0.212     36         1
1            5      162            104  ...                0.151     52         1
2            2      197             70  ...                0.575     62         1
3           13      158            114  ...                0.257     44         1
4            0      162             76  ...                0.759     25         1

[5 rows x 9 columns]
```

ii. Displaying the number of rows randomly.

```
>> data.sample()
```

```
data.sample(10)
  Pregnancies  Glucose  ... Age Outcome
35           7     106  ...  48        0
471          8       74  ...  39        0
33           1     116  ...  25        0
479          0     131  ...  26        1
60           2     100  ...  28        1
308          3       82  ...  25        0
391          6     103  ...  29        0
440          3     121  ...  25        1
461          8     179  ...  36        1
213          0     135  ...  26        0

[10 rows x 9 columns]
```

iii. Displaying the number of columns, shape and names of the columns.

```
>> data.columns
```

```
>> data.shape
```

```
data.columns
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')
print(data)
  Pregnancies  Glucose  ... Age Outcome
0           4     183  ...  36        1
1           5     162  ...  52        1
2           2     197  ...  62        1
3          13     158  ...  44        1
4           0     162  ...  25        1
..         ...     ...  ...  ...     ...
503          7     179  ...  60        0
504          0     113  ...  23        1
505          3     128  ...  27        1
506          1     119  ...  26        0
507          3       84  ...  28        0

[508 rows x 9 columns]
```

iv. Slicing the rows

```
>> sliced_data=data[40:51]
```

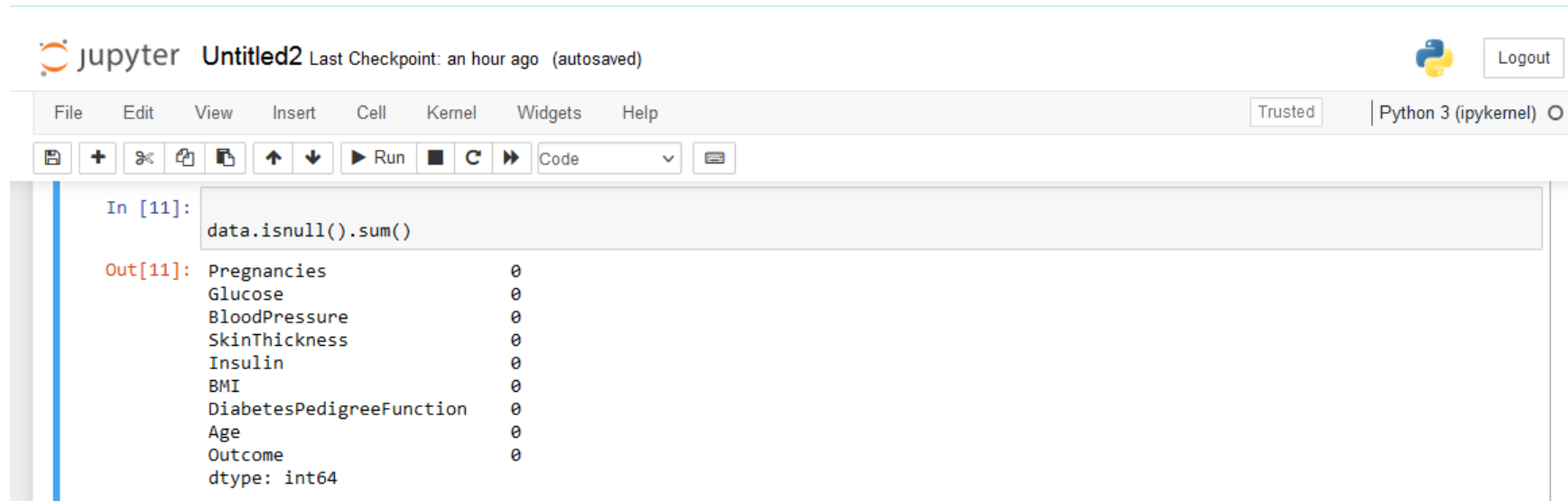
```
>>print(sliced_data
```

```
sliced_data=data[40:51]
print(sliced_data)
```

	Pregnancies	Glucose	BloodPressure	...	DiabetesPedigreeFunction	Age	Outcome
40	3	120	70	...	0.452	30	0
41	1	99	72	...	0.412	21	0
42	10	168	74	...	0.537	34	1
43	5	147	75	...	0.434	28	0
44	3	141	0	...	0.761	27	1
45	8	126	74	...	0.162	39	0
46	12	100	84	...	0.488	46	0
47	2	122	52	...	0.816	28	0
48	9	72	78	...	0.280	38	0
49	0	145	0	...	0.630	31	1
50	6	144	72	...	0.255	40	0

```
[11 rows x 9 columns]
```

v. Slicing the rows



The image shows a Jupyter Notebook interface. At the top, the header includes the Jupyter logo, the text "Untitled2", and "Last Checkpoint: an hour ago (autosaved)". On the right, there is a Python logo and a "Logout" button. Below the header is a menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. To the right of the menu bar, it says "Trusted" and "Python 3 (ipykernel)". Below the menu bar is a toolbar with icons for saving, adding cells, undo, redo, and running code. The main area contains a code cell with the input `data.isnull().sum()`. The output of this cell is displayed below it, showing a series of variables and their corresponding null counts, all of which are 0.

```
In [11]: data.isnull().sum()

Out[11]: Pregnancies      0
         Glucose          0
         BloodPressure    0
         SkinThickness    0
         Insulin          0
         BMI              0
         DiabetesPedigreeFunction  0
         Age              0
         Outcome          0
         dtype: int64
```

vi. Data Description

jupyter Untitled2 Last Checkpoint: an hour ago (autosaved)  Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

In [12]: `data.describe()`

Out[12]:

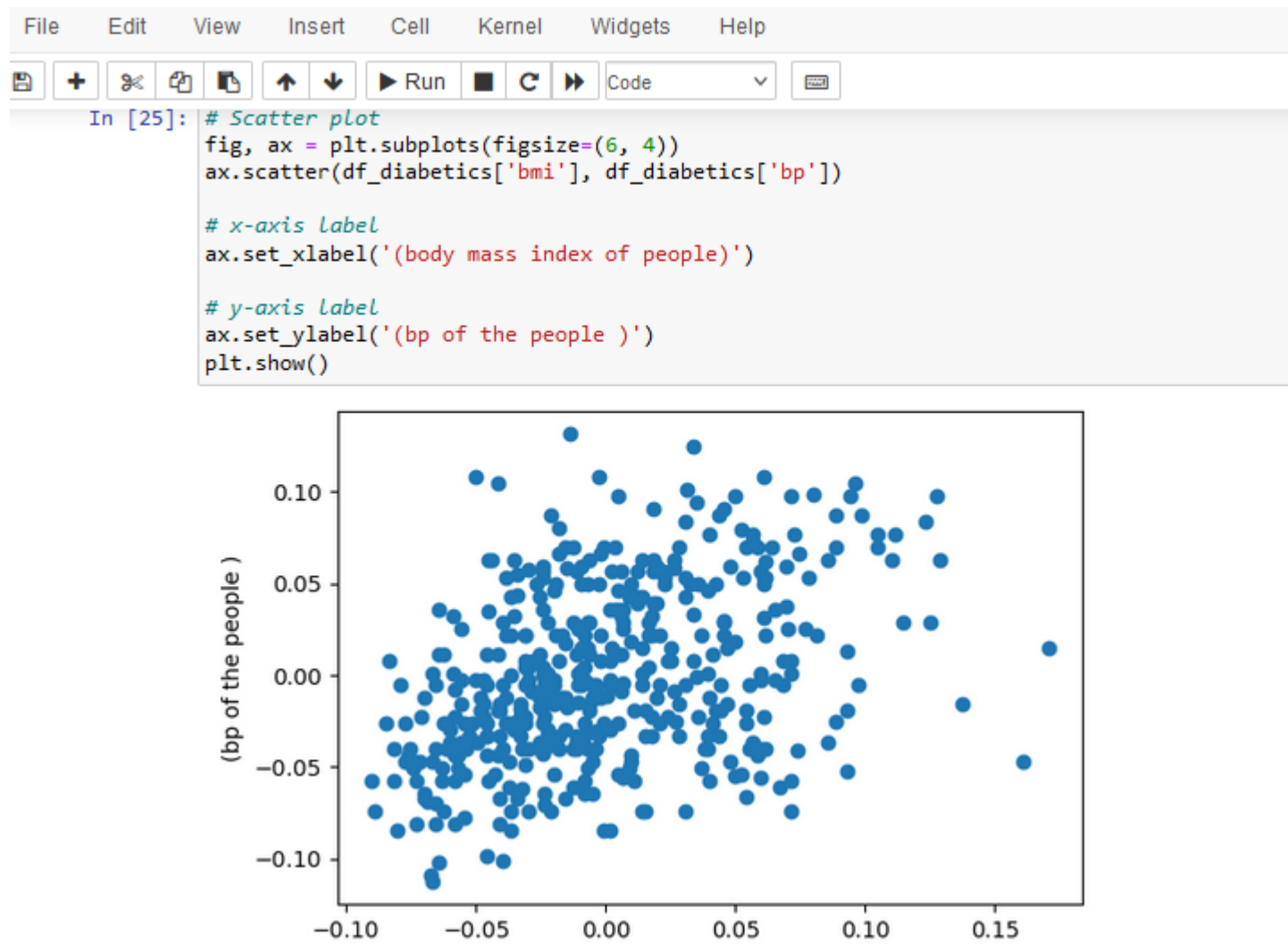
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	508.000000	508.000000	508.000000	508.000000	508.000000	508.000000	508.000000	508.000000	508.000000
mean	3.913386	120.911417	68.474409	19.566929	80.232283	31.764173	0.460567	33.346457	0.344488
std	3.305064	31.786424	20.436203	16.275653	119.686281	8.072185	0.322653	11.768060	0.475670
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	100.000000	62.000000	0.000000	0.000000	26.600000	0.248000	24.000000	0.000000
50%	3.000000	117.000000	71.000000	22.000000	0.000000	32.000000	0.375000	29.000000	0.000000
75%	6.000000	140.000000	80.000000	32.000000	127.250000	36.625000	0.591500	41.000000	1.000000
max	14.000000	198.000000	122.000000	99.000000	846.000000	59.400000	2.420000	81.000000	1.000000

In [13]: `import sklearn`

In [14]: `from sklearn.datasets import load_diabetes`

In [15]: `import pandas as pd`

vii. Scatter plot some portion of diabetes dataset.



viii. Upper bound and lower bound some portions of diabetes dataset.

```
File Edit View Insert Cell Kernel Widgets Help
+ ✂ 📄 📁 ⬆ ⬇ ▶ Run ■ ↺ ⬆ Code ▼ 🗨
```

```
In [28]: # IQR
Q1 = np.percentile(df_diabetics['bmi'], 25, method='midpoint')
Q3 = np.percentile(df_diabetics['bmi'], 75, method='midpoint')
IQR = Q3 - Q1
print(IQR)

0.06520763046978838

In [29]: # Above Upper bound
upper = Q3+1.5*IQR
upper_array = np.array(df_diabetics['bmi'] >= upper)
print("Upper Bound:", upper)
print(upper_array.sum())

# Below Lower bound
lower = Q1-1.5*IQR
lower_array = np.array(df_diabetics['bmi'] <= lower)
print("Lower Bound:", lower)
print(lower_array.sum())

Upper Bound: 0.12879000811776306
3
Lower Bound: -0.13204051376139045
0
```

xi. Load the some portions of diabetes dataset in frame.

File Edit View Insert Cell Kernel Widgets Help Trusted



```
In [46]: import pandas as pd
         from sklearn.datasets import load_diabetes
         import seaborn as sns
         import matplotlib.pyplot as plt
```

```
In [47]: # Load the dataset with frame
         df = load_diabetes(as_frame=True)
         # convert into pandas dataframe
         df = df.frame
         # Print first 5 rows
         df.head()
```

Out[47]:

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

x. Load the some portions of diabetes dataset in correlation matrix.

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

```
In [48]: # Find the pearson correlations matrix
corr = df.corr(method = 'pearson')
corr
```

Out[48]:

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
age	1.000000	0.173737	0.185085	0.335428	0.260061	0.219243	-0.075181	0.203841	0.270774	0.301731	0.187889
sex	0.173737	1.000000	0.088161	0.241010	0.035277	0.142637	-0.379090	0.332115	0.149916	0.208133	0.043062
bmi	0.185085	0.088161	1.000000	0.395411	0.249777	0.261170	-0.366811	0.413807	0.446157	0.388680	0.586450
bp	0.335428	0.241010	0.395411	1.000000	0.242464	0.185548	-0.178762	0.257650	0.393480	0.390430	0.441482
s1	0.260061	0.035277	0.249777	0.242464	1.000000	0.896663	0.051519	0.542207	0.515503	0.325717	0.212022
s2	0.219243	0.142637	0.261170	0.185548	0.896663	1.000000	-0.196455	0.659817	0.318357	0.290600	0.174054
s3	-0.075181	-0.379090	-0.366811	-0.178762	0.051519	-0.196455	1.000000	-0.738493	-0.398577	-0.273697	-0.394789
s4	0.203841	0.332115	0.413807	0.257650	0.542207	0.659817	-0.738493	1.000000	0.617859	0.417212	0.430453
s5	0.270774	0.149916	0.446157	0.393480	0.515503	0.318357	-0.398577	0.617859	1.000000	0.464669	0.565883
s6	0.301731	0.208133	0.388680	0.390430	0.325717	0.290600	-0.273697	0.417212	0.464669	1.000000	0.382483
target	0.187889	0.043062	0.586450	0.441482	0.212022	0.174054	-0.394789	0.430453	0.565883	0.382483	1.000000

xi. Find the correlations using numpy between two columns

```
In [49]: # correaltions between age and sex columns  
c = np.corrcoef(df['age'],df['sex'])  
print('Correlations between age and sex\n',c)
```

```
Correlations between age and sex  
[[1.         0.1737371]  
 [0.1737371 1.         ]]
```

```
In [50]: plt.figure(figsize=(10,8), dpi =500)  
sns.heatmap(corr,annot=True,fmt=".2f", linewidth=.5)  
plt.show()
```

xii. Plot the correlation matrix with the seaborn heatmap.

```
in [50]: plt.figure(figsize=(10,8), dpi =100)  
sns.heatmap(corr,annot=True,fmt=".2F", linewidth=.5)  
plt.show()
```



NumPy: Numpy is a library for the python programming language.

Pandas: Used for Data manipulation and Analysis.

Iloc: iloc is a property of the pandas.DataFrame class in python's pandas library.

Step-8: Implementation of K-Means Clustering of Iris Dataset

<< Resourec link: [K-Means Clustering of Iris Dataset | Kaggle](#)

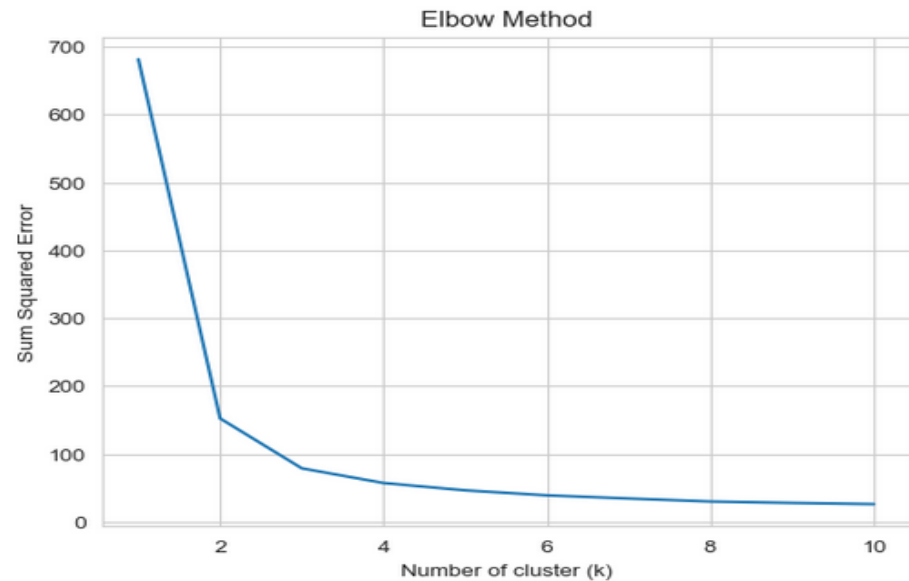
i. Using the elbow method to determine the optimal number of clusters for k-means clustering.

```
In [6]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
```

```
In [7]: sns.set_style("whitegrid")
g=sns.lineplot(x=range(1,11), y=sse)

g.set(xlabel ="Number of cluster (k)",
      ylabel = "Sum Squared Error",
      title ='Elbow Method')

plt.show()
```

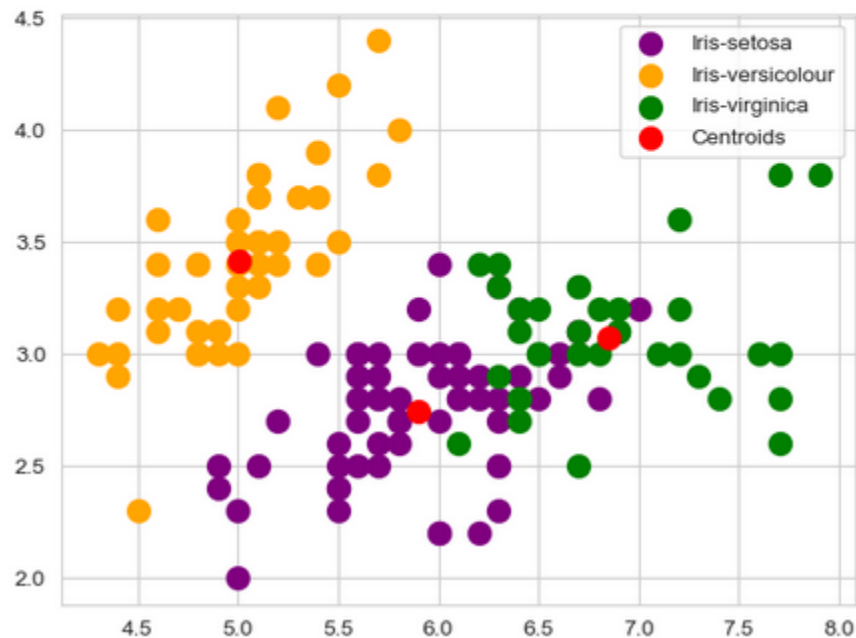


ii. Visualizing the cluster.

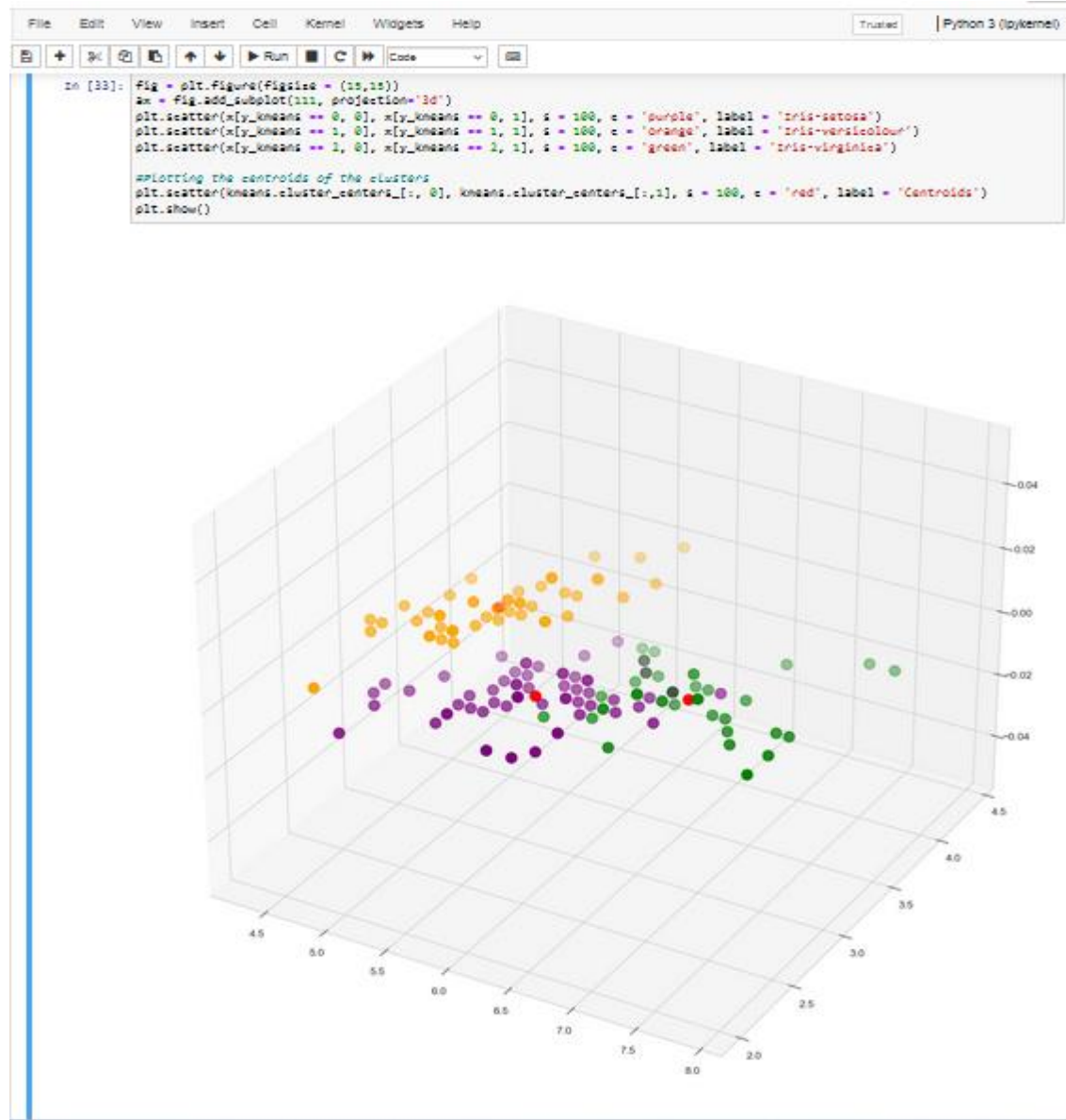
```
In [32]: #Visualising the clusters
plt.scatter(x[y_kmeans == 0, 0], x[y_kmeans == 0, 1], s = 100, c = 'purple', label = 'Iris-setosa')
plt.scatter(x[y_kmeans == 1, 0], x[y_kmeans == 1, 1], s = 100, c = 'orange', label = 'Iris-versicolour')
plt.scatter(x[y_kmeans == 2, 0], x[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Iris-virginica')

#Plotting the centroids of the clusters
plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s = 100, c = 'red', label = 'Centroids')
plt.legend()
```

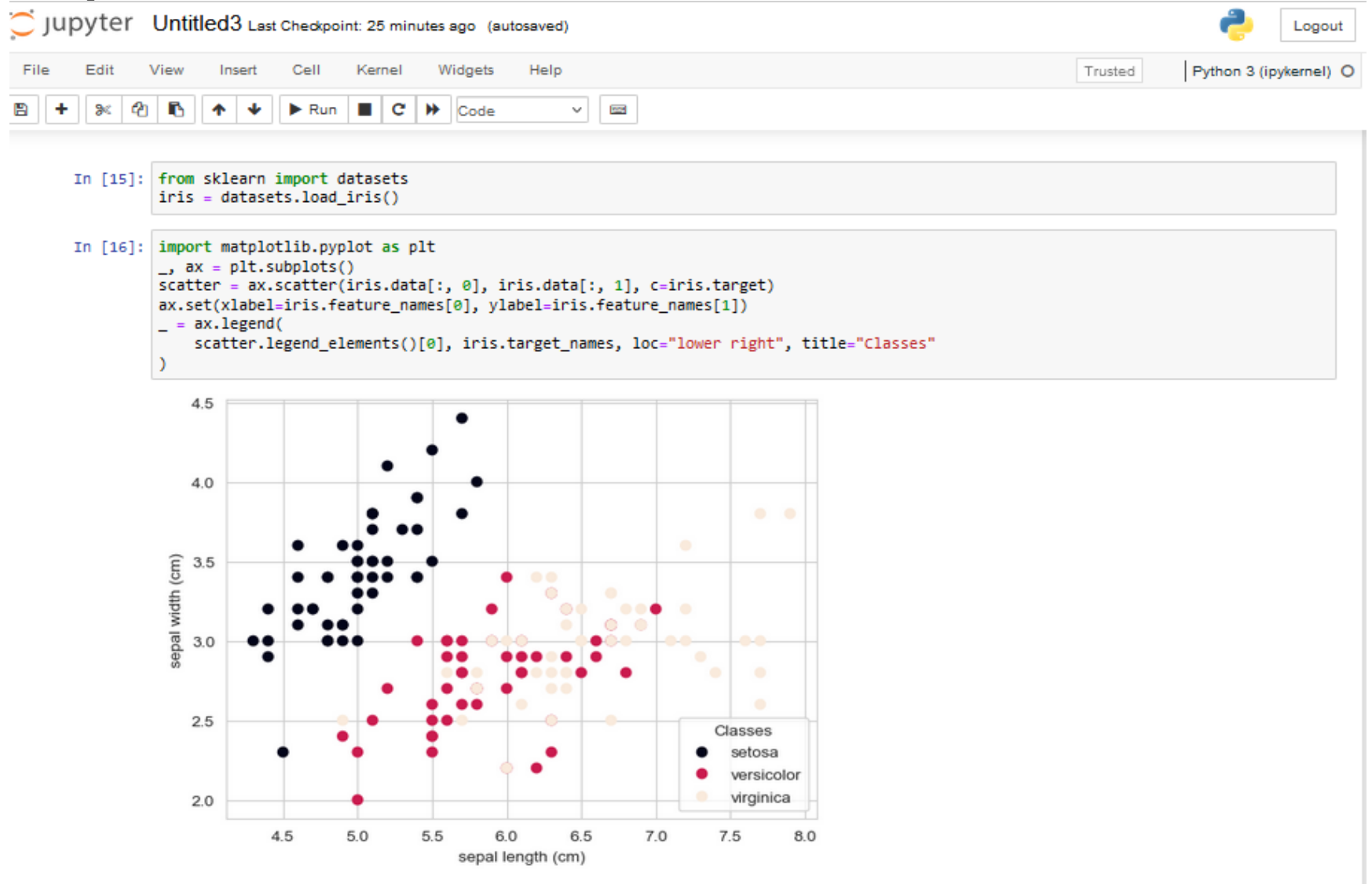
Out[32]: <matplotlib.legend.Legend at 0x204a185d1d0>



iii. 3D scatterplot using matplotlib.



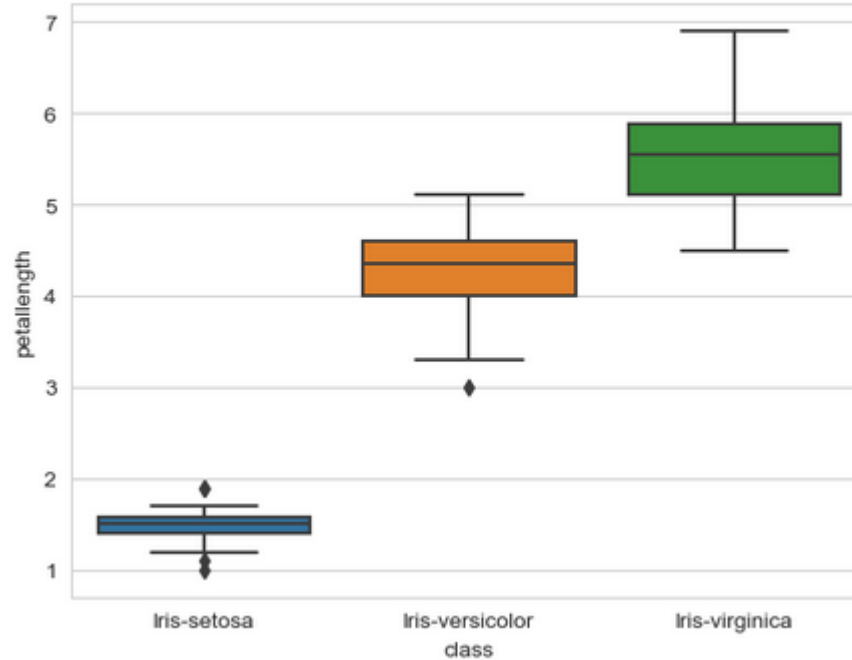
iv. Some portion of Scatter Plot of the Iris dataset



Note: Each point in the scatter plot refers to one of the 150 iris flowers in the dataset, with the color indicating their respective type (Setosa, Versicolour, and Virginica). You can already see a pattern regarding the Setosa type, which is easily identifiable based on its short and wide sepal. Only considering these 2 dimensions, sepal width and length, there's still overlap between the Versicolor and Virginica types.

v. Some portion of Box Plot of Iris dataset.

```
In [27]: sns.boxplot(x="class",y="petallength",data=iris)
plt.show()
```

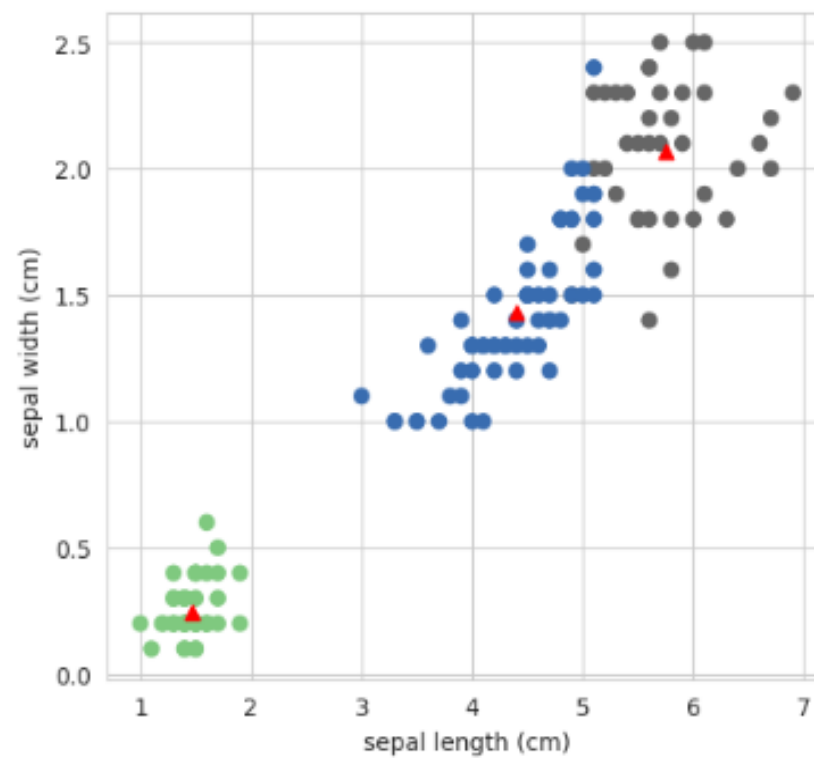
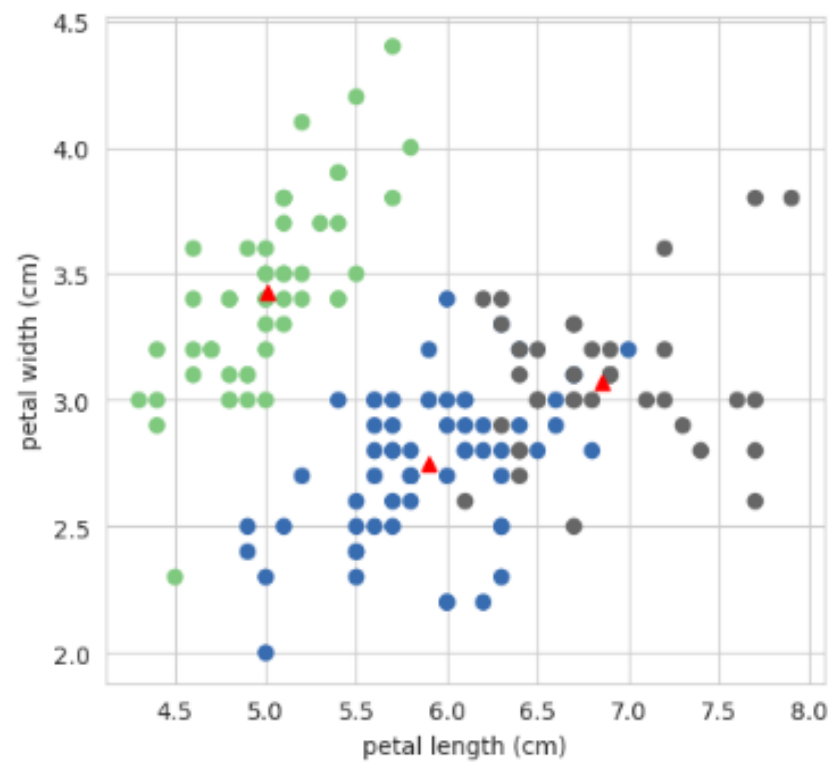


vi. Some portion of k-means cluster of Iris dataset.

```
In [12]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
```

```
In [13]: plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.scatter(X[:,0],X[:,1],c = red, cmap=cm.Accent)
plt.grid(True)
for center in kmeans.cluster_centers_:
    center = center[:2]
    plt.scatter(center[0],center[1],marker = '^',c = 'red')
plt.xlabel("petal length (cm)")
plt.ylabel("petal width (cm)")

plt.subplot(1,2,2)
plt.scatter(X[:,2],X[:,3],c = red, cmap=cm.Accent)
plt.grid(True)
for center in kmeans.cluster_centers_:
    center = center[2:4]
    plt.scatter(center[0],center[1],marker = '^',c = 'red')
plt.xlabel("sepal length (cm)")
plt.ylabel("sepal width (cm)")
plt.show()
```



Step-9: Implementation of Apriori and Clustering algorithm using WEKA.

- i. Open the weka explorer
- ii. Open the preprocessed data (iris.arff) from the install location.
- iii. Opening ARFF file in WEKA explorer

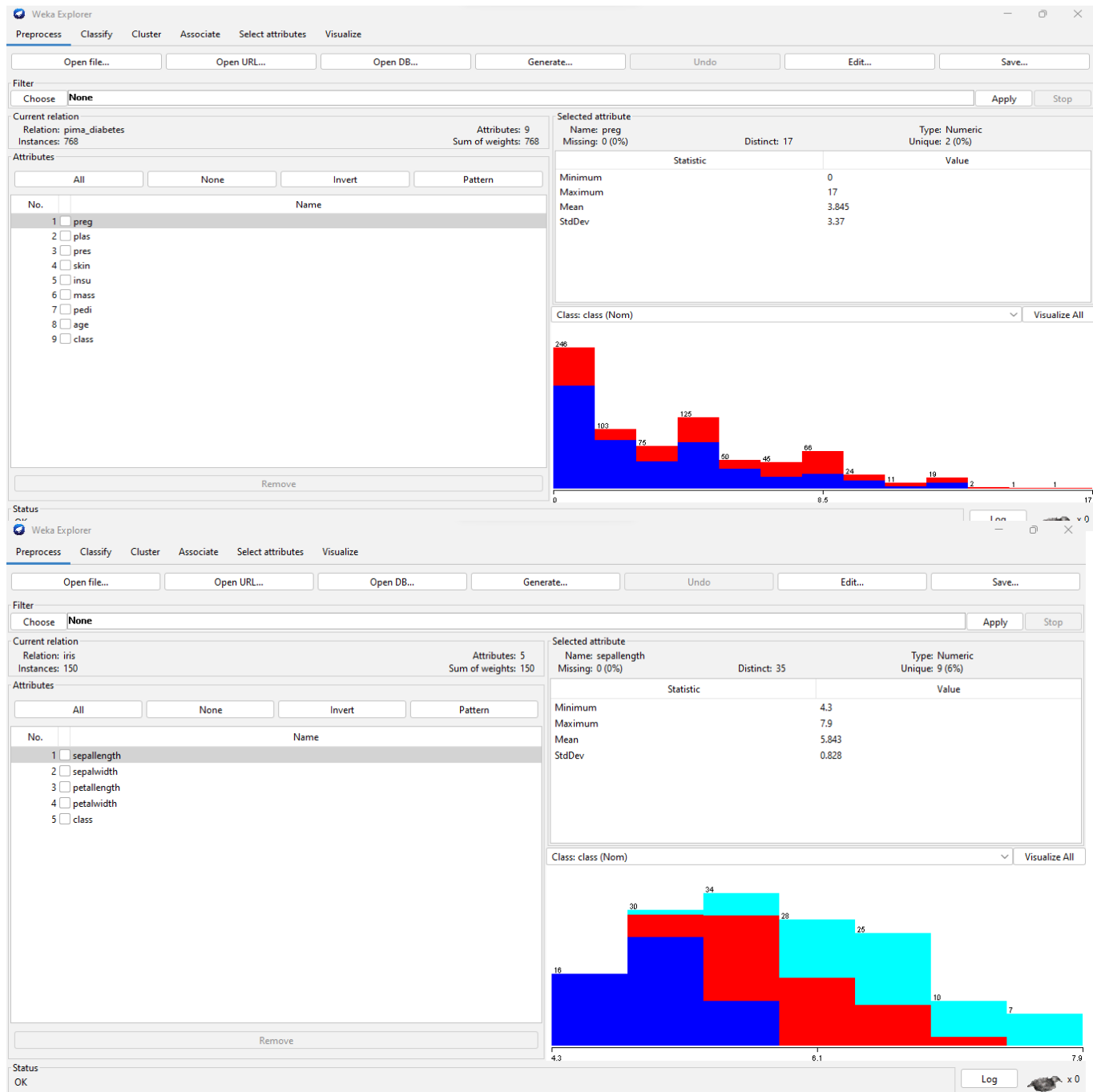
>>Weka Explorer-->Preprocess-->Generate

Diabetes and Iris dataset an ARFF file

```
Generated Instances (incl. comments)
%% Commandline%% weka.datagenrators.classifiers.classification.RDGL -S 1 -n 100 -a 10 -c 2 -N 0 -I 0 -M 1 -R 10%relation weka.datagenrators.classifiers.classification.RDGL-S_1_-n_100_-a_10_

@attribute a0 {false,true}
@attribute a1 {false,true}
@attribute a2 {false,true}
@attribute a3 {false,true}
@attribute a4 {false,true}
@attribute a5 {false,true}
@attribute a6 {false,true}
@attribute a7 {false,true}
@attribute a8 {false,true}
@attribute a9 {false,true}
@attribute class {c0,c1}

@data
true,false,false,false,true,false,true,true,true,true,c0
true,true,false,false,false,true,true,false,true,true,c0
false,false,true,true,false,true,false,false,false,false,c1
false,true,false,true,true,false,true,true,false,false,c0
true,true,false,false,false,true,true,false,false,true,c0
true,true,true,true,false,false,true,false,true,true,c1
true,true,false,true,true,false,true,false,false,true,c0
false,false,true,false,true,true,false,true,false,true,c1
true,true,false,true,false,true,false,true,true,true,c0
false,false,true,true,true,true,true,true,false,true,c1
false,true,false,false,true,true,false,true,false,true,c0
true,false,true,false,true,false,false,false,false,true,c0
true,true,false,true,true,true,false,false,false,false,c1
false,false,false,true,true,true,false,false,true,false,c1
false,false,false,true,true,false,false,false,true,false,c0
true,false,false,true,false,false,true,false,false,false,c0
true,true,true,false,false,true,true,true,false,true,c1
true,false,false,true,false,true,false,true,false,false,c0
true,false,true,false,false,false,false,false,true,true,c0
false,false,false,true,false,true,true,false,false,true,c1
false,true,false,true,true,true,false,true,true,false,c1
false,true,false,true,true,false,false,false,false,true,c0
false,true,true,true,false,false,false,false,true,true,c0
false,true,false,false,true,false,false,false,false,c0
true,true,true,false,false,true,false,false,true,true,c0
```



iv. Apply the Apriori algorithm in WEKA

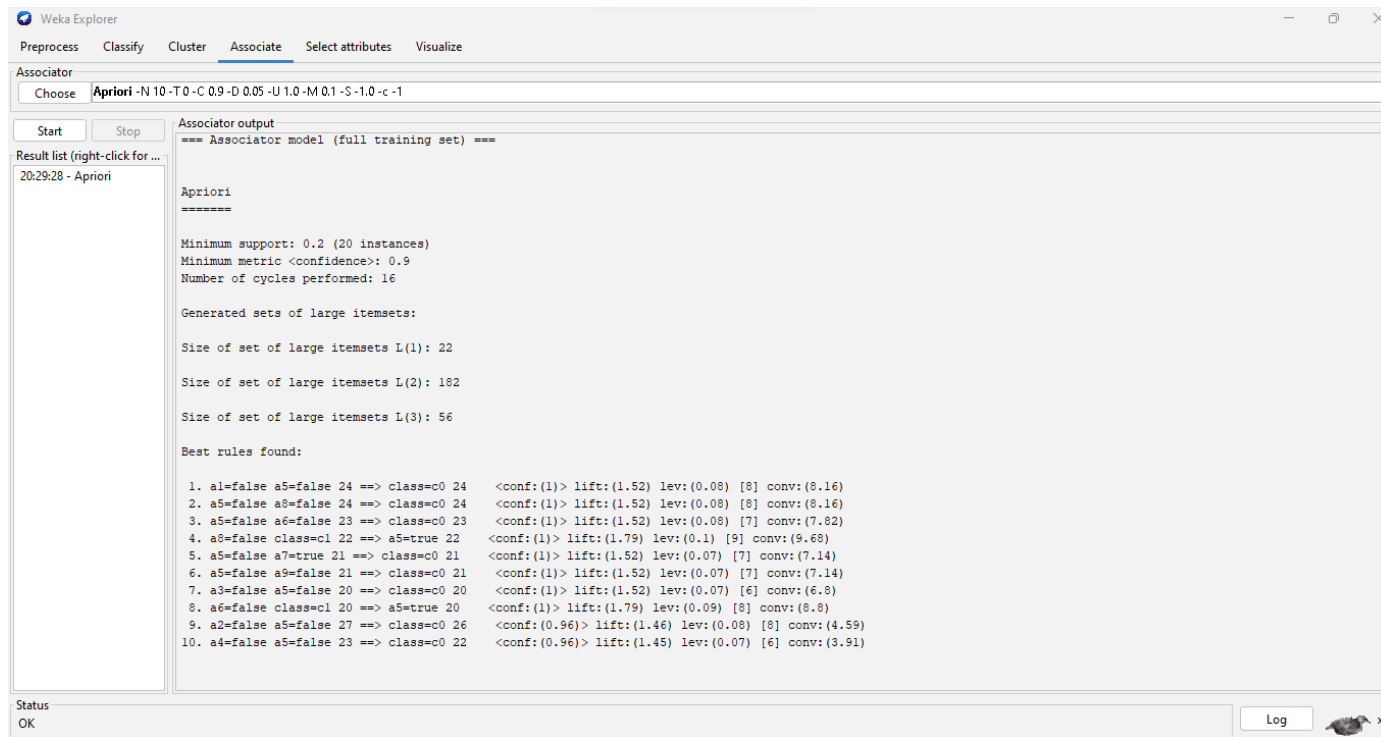


Fig: Apply the Apriori algorithm in WEKA

v. Apply the EM clustering algorithm in WEKA

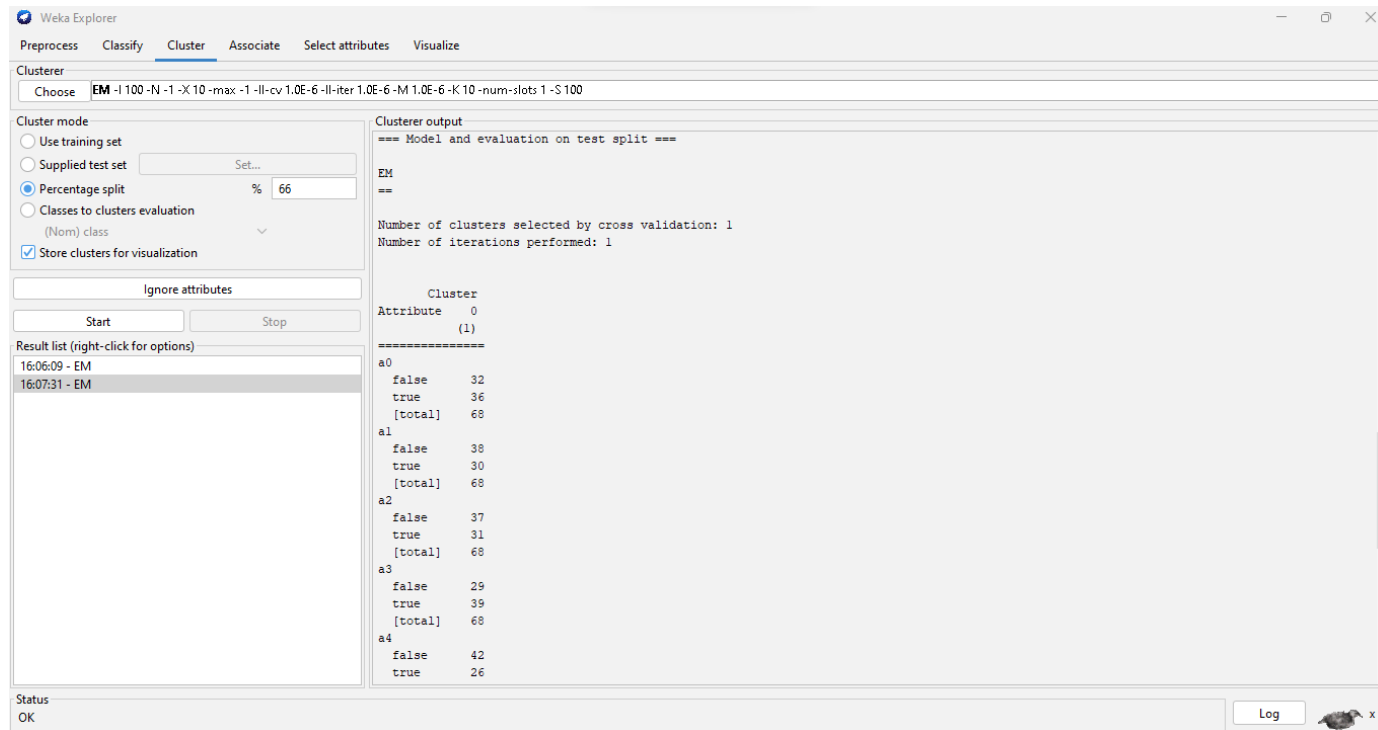


Fig: Apply the EM clustering algorithm in WEKA

vi. Iris dataset run info in WEKA

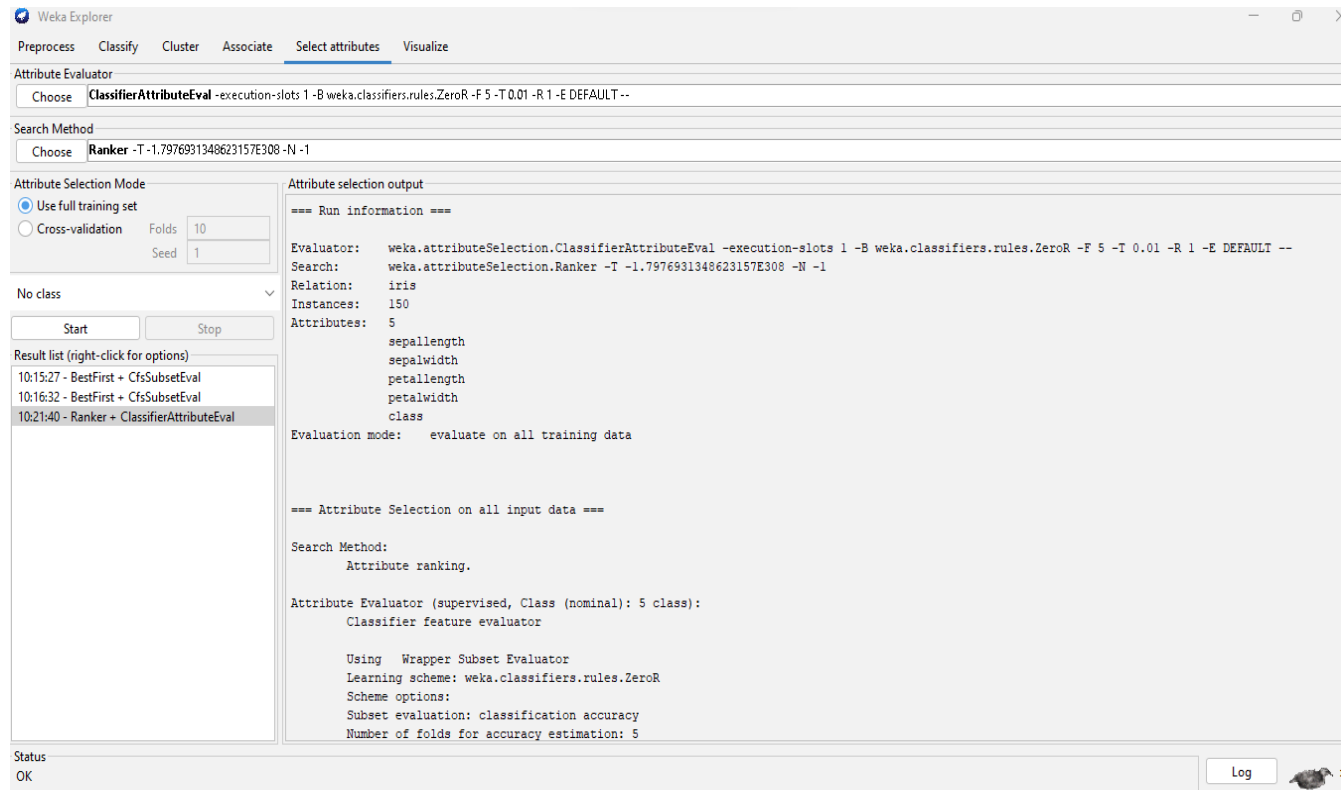


Fig: Iris dataset run info in WEKA

vii. Diabetes dataset run info in WEKA

The screenshot shows the Weka Explorer window with the 'Select attributes' tab selected. The 'Attribute Evaluator' is set to 'PrincipalComponents -R 0.95 -A 5' and the 'Search Method' is set to 'Ranker -T -1.7976931348623157E308 -N -1'. The 'Attribute Selection Mode' is set to 'Use full training set' with 10 folds and seed 1. The 'Result list' shows three entries, with the third entry '16:32:26 - Ranker + PrincipalComponents' selected. The 'Attribute selection output' pane displays the following information:

Attributes: 9
 preg
 plas
 pres
 skin
 insu
 mass
 pedi
 age
 class

Evaluation mode: evaluate on all training data

=== Attribute Selection on all input data ===

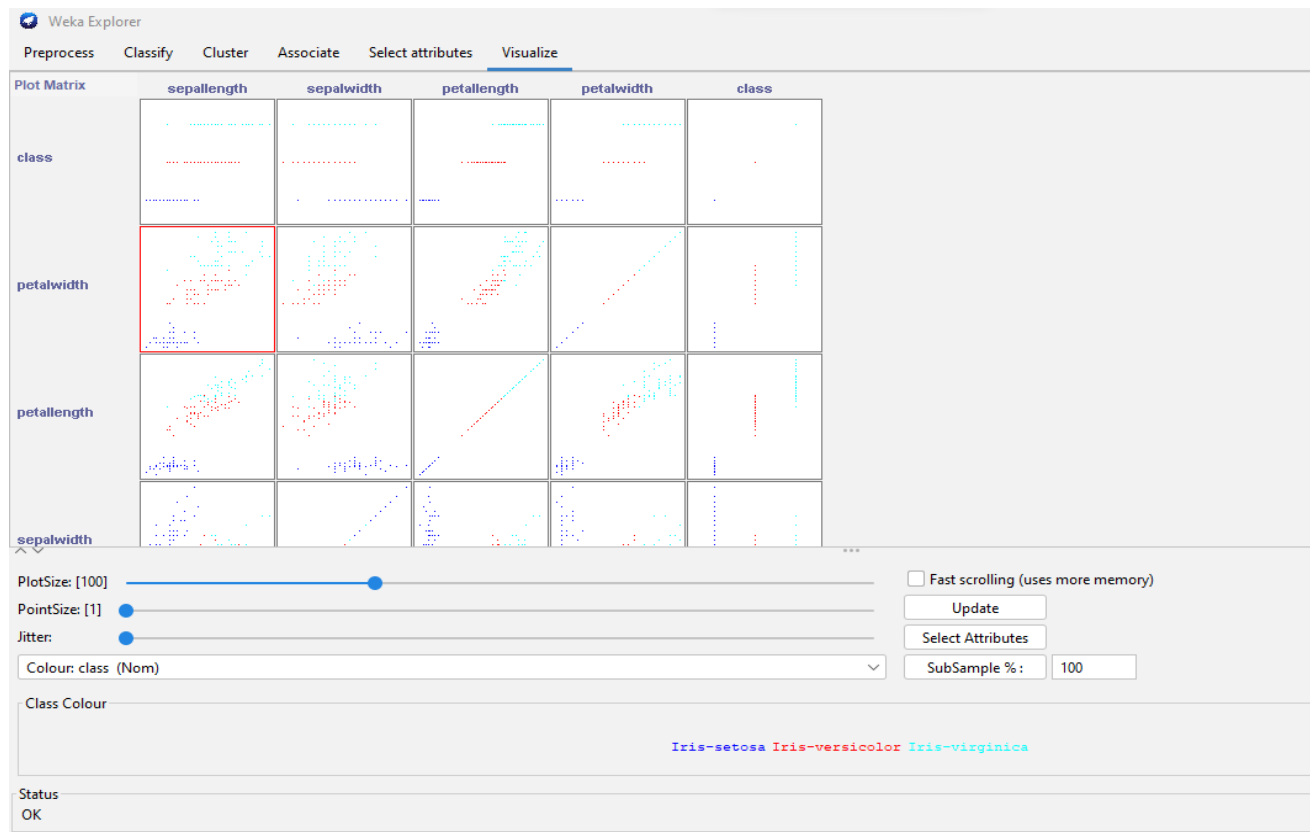
Search Method:
 Attribute ranking.

Attribute Evaluator (unsupervised):
 Principal Components Attribute Transformer

Correlation matrix

1	0.13	0.14	-0.08	-0.07	0.02	-0.03	0.54	0.22
0.13	1	0.15	0.06	0.33	0.22	0.14	0.26	0.47
0.14	0.15	1	0.21	0.09	0.28	0.04	0.24	0.07
-0.08	0.06	0.21	1	0.44	0.39	0.18	-0.11	0.07
-0.07	0.33	0.09	0.44	1	0.2	0.19	-0.04	0.13
0.02	0.22	0.28	0.39	0.2	1	0.14	0.04	0.29
-0.03	0.14	0.04	0.18	0.19	0.14	1	0.03	0.17

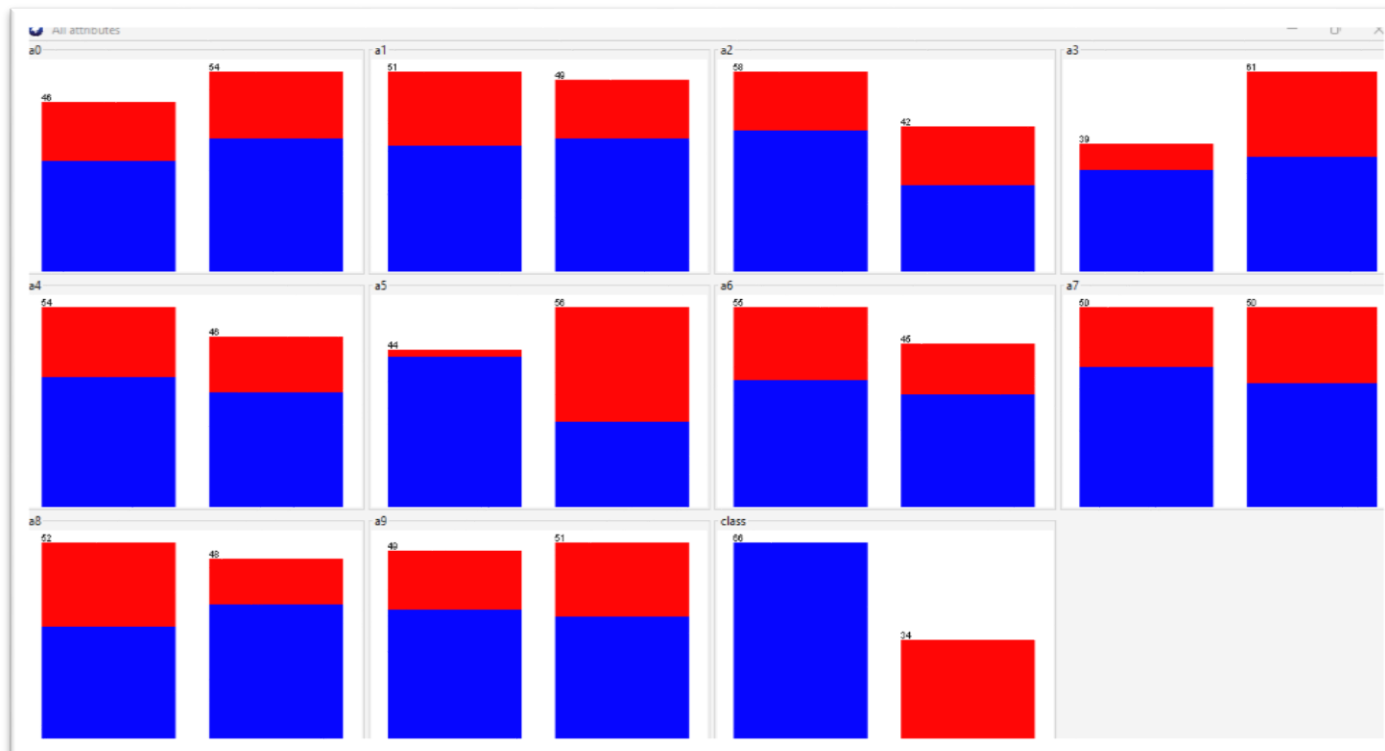
viii. Iris dataset plot matrix representation in WEKA



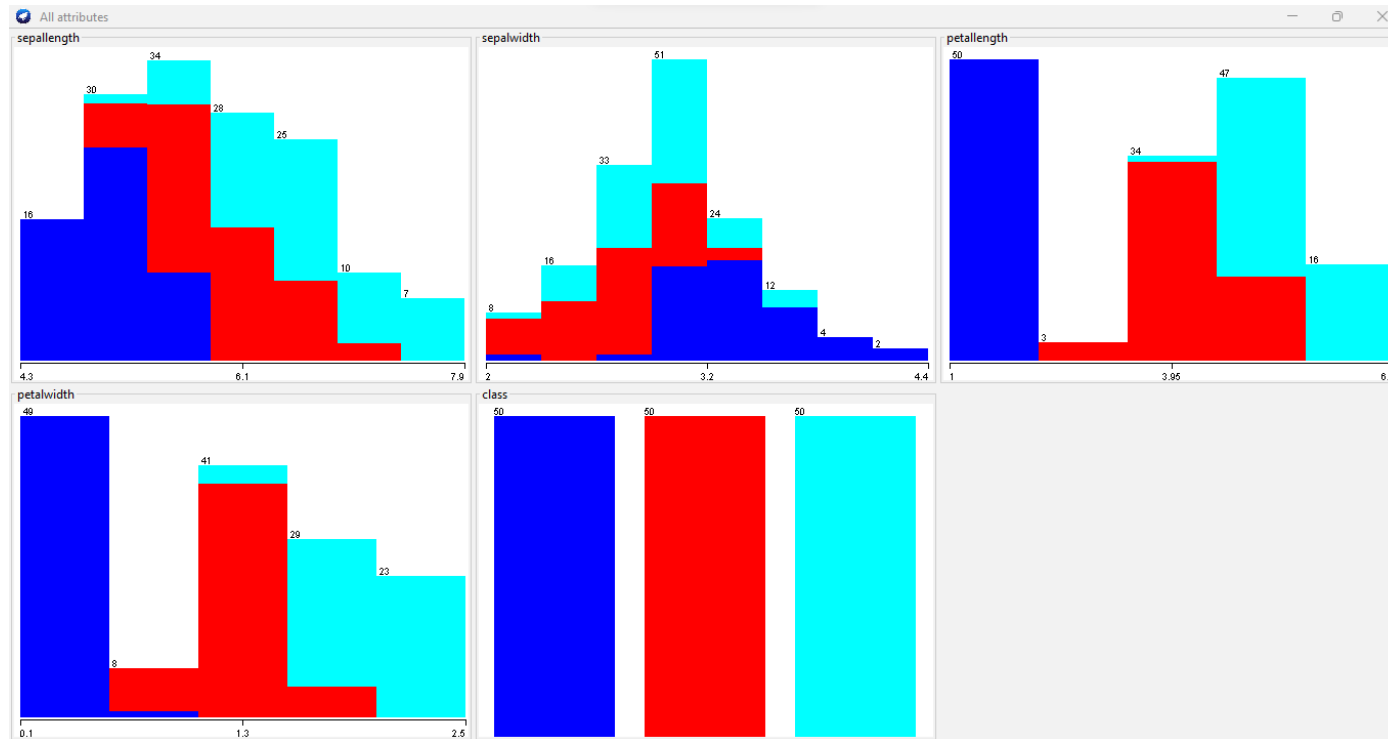
ix. Diabetes dataset plot matrix representation in WEKA



x. Diabetes dataset visual representation in WEKA



xi. Iris dataset visual representation in WEKA



xii. Iris dataset relation in WEKA

Viewer					
Relation: iris					
No.	1: sepalength Numeric	2: sepalwidth Numeric	3: petallength Numeric	4: petalwidth Numeric	5: class Nominal
1	5.1	3.5	1.4	0.2	Iris-se...
2	4.9	3.0	1.4	0.2	Iris-se...
3	4.7	3.2	1.3	0.2	Iris-se...
4	4.6	3.1	1.5	0.2	Iris-se...
5	5.0	3.6	1.4	0.2	Iris-se...
6	5.4	3.9	1.7	0.4	Iris-se...
7	4.6	3.4	1.4	0.3	Iris-se...
8	5.0	3.4	1.5	0.2	Iris-se...
9	4.4	2.9	1.4	0.2	Iris-se...
10	4.9	3.1	1.5	0.1	Iris-se...
11	5.4	3.7	1.5	0.2	Iris-se...
12	4.8	3.4	1.6	0.2	Iris-se...
13	4.8	3.0	1.4	0.1	Iris-se...
14	4.3	3.0	1.1	0.1	Iris-se...
15	5.8	4.0	1.2	0.2	Iris-se...
16	5.7	4.4	1.5	0.4	Iris-se...
17	5.4	3.9	1.3	0.4	Iris-se...
18	5.1	3.5	1.4	0.3	Iris-se...
19	5.7	3.8	1.7	0.3	Iris-se...
20	5.1	3.8	1.5	0.3	Iris-se...
21	5.4	3.4	1.7	0.2	Iris-se...
22	5.1	3.7	1.5	0.4	Iris-se...
23	4.6	3.6	1.0	0.2	Iris-se...
24	5.1	3.3	1.7	0.5	Iris-se...

xiii. Diabetes dataset relation in WEKA

Viewer									
Relation: pima_diabetes									
No.	1: preg Numeric	2: plas Numeric	3: pres Numeric	4: skin Numeric	5: insu Numeric	6: mass Numeric	7: pedi Numeric	8: age Numeric	9: class Nominal
1	6.0	148.0	72.0	35.0	0.0	33.6	0.627	50.0	tested...
2	1.0	85.0	66.0	29.0	0.0	26.6	0.351	31.0	tested...
3	8.0	183.0	64.0	0.0	0.0	23.3	0.672	32.0	tested...
4	1.0	89.0	66.0	23.0	94.0	28.1	0.167	21.0	tested...
5	0.0	137.0	40.0	35.0	168.0	43.1	2.288	33.0	tested...
6	5.0	116.0	74.0	0.0	0.0	25.6	0.201	30.0	tested...
7	3.0	78.0	50.0	32.0	88.0	31.0	0.248	26.0	tested...
8	10.0	115.0	0.0	0.0	0.0	35.3	0.134	29.0	tested...
9	2.0	197.0	70.0	45.0	543.0	30.5	0.158	53.0	tested...
10	8.0	125.0	96.0	0.0	0.0	0.0	0.232	54.0	tested...
11	4.0	110.0	92.0	0.0	0.0	37.6	0.191	30.0	tested...
12	10.0	168.0	74.0	0.0	0.0	38.0	0.537	34.0	tested...
13	10.0	139.0	80.0	0.0	0.0	27.1	1.441	57.0	tested...
14	1.0	189.0	60.0	23.0	846.0	30.1	0.398	59.0	tested...
15	5.0	166.0	72.0	19.0	175.0	25.8	0.587	51.0	tested...
16	7.0	100.0	0.0	0.0	0.0	30.0	0.484	32.0	tested...
17	0.0	118.0	84.0	47.0	230.0	45.8	0.551	31.0	tested...
18	7.0	107.0	74.0	0.0	0.0	29.6	0.254	31.0	tested...
19	1.0	103.0	30.0	38.0	83.0	43.3	0.183	33.0	tested...
20	1.0	115.0	70.0	30.0	96.0	34.6	0.529	32.0	tested...
21	3.0	126.0	88.0	41.0	235.0	39.3	0.704	27.0	tested...
22	8.0	99.0	84.0	0.0	0.0	35.4	0.388	50.0	tested...
23	7.0	196.0	90.0	0.0	0.0	39.8	0.451	41.0	tested...
24	9.0	119.0	80.0	35.0	0.0	29.0	0.263	29.0	tested...